Reviewer Pack — Minimal Rank of Tropicalized Deligne-Lusztig Indicators Boun…

Entry 0f2b036c337c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 14:57:02 UTC

Minimal Rank of Tropicalized Deligne-Lusztig Indicators Bounds Communication Complexity of Entanglement Distillation

Entry ID: `0f2b036c337c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 14:57:02 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Deligne-Lusztig Theory) **Field B** (complexity object): Quantum Information Theory: Entanglement Distillation

Statement:

> ['For every entangled quantum state ρ represented as a density matrix, the minimal rank of its tropicalized Deligne-Lusztig indicators is Ω(log(1/ε)), where ε is the maximal distillable entropy from ρ.', 'For all entangled states ρ with at most 40 qubits, there exists a constant c > 0 such that the minimal rank of the tropicalized Deligne-Lusztig indicators, τ(ρ), satisfies τ(ρ) ≥ c log(1/ε).', 'This lower bound holds for all ε ≥ 2−20 and the conjecture is falsified by any entangled state ρ with τ(ρ) < c log(1/2−20) for some constant c.']

Rationale (proposer's reasoning):

> ['The Deligne-Lusztig theory, which is a part of tropical geometry, provides a way to study the geometric properties of algebraic varieties. The tropicalization process can be applied to quantum states, potentially revealing their geometric structure.', 'Entanglement distillation, being a task in quantum information theory, deals with extracting highly entangled states from less entangled ones. The conjecture suggests that the complexity of this process is related to the geometric properties of the quantum state.', 'The minimal rank of tropicalized Deligne-Lusztig indicators may expose hidden structures in the quantum state that are crucial for understanding its distillability.']

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2ad2d13de08e49e7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 95% of entangled states ρ with at most 40 qubits, there exists a constant $c > 0$ such that $\tau(\rho) \geq c \log(1/\varepsilon)$ with a Spearman's rank correlation coefficient of $r \geq 0.7$, where ε is the distillable entropy and $\tau(\rho)$ is the minimal rank of tropicalized Deligne-Lusztig indicators.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND Deligne-Lusztig theory AND quantum information theory` - `entanglement distillation AND communication complexity AND tropicalized Deligne-Lusztig indicators` - `minimal rank AND tropicalization AND entanglement distillation entropy`

Top relevant hits considered: - [http://arxiv.org/abs/hep-th/9201009v1] Large-Small Equivalence in String Theory - [http://arxiv.org/abs/1503.08426v1] K3 en route From Geometry to Conformal Field Theory - [http://arxiv.org/abs/hep-th/9706226v2] F-theory, SO(32) and Toric Geometry - [http://arxiv.org/abs/2305.03489v3] No-go theorem for entanglement distillation using catalysis - [http://arxiv.org/abs/2602.09428v2] Near-optimal entanglement-communication tradeoffs for remote state preparation - [http://arxiv.org/abs/2205.08561v1] Learning Quantum Entanglement Distillation with Noisy Classical Communications - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1612.09179v3] A compact minimal space Y such that its square $Y \times Y$ is not minimal

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_entangled_state(n):
    state = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):
        state[i][i] = Fraction(1, 2)
    return state

def matrix_multiplication(A, B):
    n = len(B[0])
    result = [[Fraction(0, 1) for _ in range(n)] for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(n):
```

```
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(i + 1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if any(matrix[i][j] != Fraction(0, 1) for j in range(n)):
            rank += 1
    return rank

def distillable_entropy(state):
    n = len(state)
    v = [Fraction(1, math.sqrt(n))] * n
    Av = matrix_multiplication(state, v)
    max_value = max(abs(x) for row in Av for x in row)
    epsilon = 0.5 - max_value / 2
    return epsilon

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    state = generate_random_entangled_state(n)
    epsilon = distillable_entropy(state)
    if epsilon <= 0:
        return {
            "metric_name": "minimal_rank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "distillable_entropy_non_positive"
        }
    Av = matrix_multiplication(state, v)
    max_value = max(abs(x) for row in Av for x in row)
    tau = rank(Av)
    return {
        "metric_name": "minimal_rank",
        "metric_value": tau,
        "instances_tested": 1,
        "conjecture_holds": tau >= Fraction(1, 10) * math.log(1 / epsilon),
        "counterexample": ""
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 53))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_tau = sum(result["metric_value"] for result in results if result["conjecture_holds"]) / l
    std_tau = math.sqrt(sum((result["metric_value"] - mean_tau) ** 2 for result in results if resu
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_tau} std={std_tau} support_fraction={support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_tau} std={std_tau} support_fraction={support_fraction
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a2a754fd.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a2a754fd.py", line 67, in run_trial
    epsilon = distillable_entropy(state)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a2a754fd.py", line 57, in distillable_e
    v = [Fraction(1, math.sqrt(n))] * n
         ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13480
2	preregistration	ollama_remote	glm4:latest	0	0	6523
3	novelty	ollama_remote	glm4:latest	0	0	4737
4	novelty	ollama_remote	glm4:latest	0	0	9448
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45620
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9882
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11359
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10844
9	judge	ollama_remote	glm4:latest	0	0	11731

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 123623 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0f2b036c337c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0f2b036c337c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0f2b036c337c.tar.gz` (if generated)